WAP to Implement 2D Transformations: Translation, Rotation, Scaling, Reflection, and Shearing.ज

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>
#include<math.h>

#define PI 3.14159

void main()
{
   int gd = DETECT, gm;
   int x1 = 200, y1 = 150;
   int x2 = 150, y2 = 250;
   int x3 = 250, y3 = 250;

   int tx, ty, choice;
   float sx, sy, angle, rad, shx;

   int nx1, ny1, nx2, ny2, nx3, ny3;

   initgraph(&gd, &gm, "C:/TURBO C3/BGI");

   /* Original Triangle */
   line(x1, y1, x2, y2);
```

```
line(x2, y2, x3, y3);
line(x3, y3, x1, y1);

printf("1. Translation\n");
printf("2. Rotation\n");
printf("3. Scaling\n");
printf("4. Reflection\n");
printf("5. Shearing\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch(choice)
{
   /* TRANSLATION */
   case 1:
      printf("Enter tx and ty: ");
      scanf("%d%d", &tx, &ty);

      nx1 = x1 + tx;
      ny1 = y1 + ty;

      nx2 = x2 + tx;
      ny2 = y2 + ty;
```

```
            nx3 = x3 + tx;
            ny3 = y3 + ty;
            break;

        /* ROTATION */
        case 2:
{
    float cx, cy;

    printf("Enter rotation angle: ");
    scanf("%f", &angle);

    rad = angle * PI / 180.0;

    /* Center of triangle */
    cx = (x1 + x2 + x3) / 3.0;
    cy = (y1 + y2 + y3) / 3.0;

    /* Rotate first point */
    nx1 = cx + (x1 - cx) * cos(rad)
            - (y1 - cy) * sin(rad);

    ny1 = cy + (x1 - cx) * sin(rad)
            + (y1 - cy) * cos(rad);
```

```
    /* Rotate second point */
    nx2 = cx + (x2 - cx) * cos(rad)
            - (y2 - cy) * sin(rad);

    ny2 = cy + (x2 - cx) * sin(rad)
            + (y2 - cy) * cos(rad);

    /* Rotate third point */
    nx3 = cx + (x3 - cx) * cos(rad)
            - (y3 - cy) * sin(rad);

    ny3 = cy + (x3 - cx) * sin(rad)
            + (y3 - cy) * cos(rad);

    break;
}

        /* SCALING */
        case 3:
{
    float cx, cy;

    printf("Enter sx and sy: ");
    scanf("%f%f", &sx, &sy);
```

```
    /* Triangle center */
    cx = (x1 + x2 + x3) / 3.0;
    cy = (y1 + y2 + y3) / 3.0;

    nx1 = cx + (x1 - cx) * sx;
    ny1 = cy + (y1 - cy) * sy;

    nx2 = cx + (x2 - cx) * sx;
    ny2 = cy + (y2 - cy) * sy;

    nx3 = cx + (x3 - cx) * sx;
    ny3 = cy + (y3 - cy) * sy;

    break;
}

      /* REFLECTION ABOUT X-AXIS */
     case 4:
{
    float cy;

    /* Center of triangle */
    cy = (y1 + y2 + y3) / 3.0;

    /* Reflection about horizontal center line */
```

```
    nx1 = x1;
    ny1 = 2 * cy - y1;

    nx2 = x2;
    ny2 = 2 * cy - y2;

    nx3 = x3;
    ny3 = 2 * cy - y3;

    break;
}

      /* X-SHEARING */
      case 5:
{
    float cy;

    printf("Enter X shear factor: ");
    scanf("%f", &shx);

    cy = (y1 + y2 + y3) / 3.0;

    nx1 = x1 + shx * (y1 - cy);
    ny1 = y1;
```

```
    nx2 = x2 + shx * (y2 - cy);
    ny2 = y2;

    nx3 = x3 + shx * (y3 - cy);
    ny3 = y3;

    break;
}
      default:
         printf("Invalid Choice");
         getch();
         closegraph();
         return;
    }

    /* Transformed Triangle */
    line(nx1, ny1, nx2, ny2);
    line(nx2, ny2, nx3, ny3);
    line(nx3, ny3, nx1, ny1);

    getch();
    closegraph();
}
```